Dental Booking Platform — V1 Scope & Architecture Plan (Shareable)

Date: 2026-06-02
Audience: Product/Engineering stakeholders
Status: Draft (agreed direction; some details pending)

1) Goal (updated scope)

Evolve the current "Dental Booking Platform" from a booking app that books into **Oryx** into a **multi-tenant platform** that:

- Supports **multiple external PMS systems** (Oryx first; others later) via an adapter layer
- Also supports an **internal lightweight PMS mode** (for clinics without a PMS), using an Oryx-style schema
- Provides a **voice layer** (Connect + Lex) and optional web booking that can be used by any tenant regardless of PMS
- Stores booking and conversation data in our platform DB for audit, analytics, and backup; and for internal PMS tenants, acts as system-of-record

2) Tenancy model (V1)

- **Tenant**: one clinic/practice (single location in V1)
- **No DSO / parent org hierarchy** in V1
- Tenants may have:
  - **External PMS mode** (e.g., Oryx)
  - **Internal PMS mode** (our DB is source-of-truth)

3) System-of-record rule (per tenant)

We explicitly support two modes:

A) External-PMS tenant (e.g., Oryx)

- **External PMS is source-of-truth** for appointments.
- Our platform persists:
  - Booking request/intents (including idempotency keys)
  - Conversation/call logs
  - A mirrored "appointment record" (at minimum: time, provider/operatory identifiers, status, external appointment id, timestamps, error state)

B) Internal-PMS tenant

- **Our DB is source-of-truth** for appointments and scheduling (clinical/billing come later).

> *Open decision (pending team discussion):* ***How much appointment detail*** *should be mirrored/stored for external-PMS tenants beyond minimal metadata.*

4) V1 product scope (recommended and agreed direction)

V1 (build now)

1. **Multi-tenant management**

    - Tenants/organizations (single clinic in V1)
    - Users
    - Roles & permissions
2. **Patient management**
3. **Scheduling & appointment management**
4. **Service catalog**
5. **Voice AI configuration**
6. **Call & conversation logs**

V1.5 (after validation)

- Clinical charting

V2

- Billing (only after scheduling is solid)

V3

- Insurance / claims

5) Booking/write behavior (external PMS)

Booking confirmation strategy (V1)

- **Synchronous booking**: "wait for Oryx success" before confirming.
- **Timeout target: 10 seconds** end-to-end for voice booking confirmation.
    - If the external PMS call does not succeed within this time budget, the booking is treated as failed for the caller (while still logging the attempt).

Idempotency (must-have)

To prevent double booking due to retries (voice platforms, user resubmits), V1 requires:

- An **idempotency key** per booking request
- Safe replays: same request → returns same outcome when feasible

6) Channel boundaries (voice + web)

Voice and web channels should **not** write directly to PMS tables. They submit requests to a single **Booking Service**, which:

- Validates inputs and tenant configuration
- Checks availability rules
- Delegates booking to the correct adapter:
    - `OryxAdapter` (external)
    - `InternalPmsAdapter` (internal PMS mode)
    - Future adapters (Open Dental, Dentrix, etc.)
- Persists booking attempts and outcomes in the platform DB

7) Architecture (service split)

Hard boundary (agreed)

- **Browser → Next.js only**
- **Next.js → FastAPI** via service-to-service authentication

Responsibilities

Next.js

- Admin portal UI
- Client portal UI
- Settings UI
- User sessions (cookies), UI auth, CSRF protections
- BFF-style endpoints that proxy/aggregate calls to FastAPI

FastAPI

- Core business logic (domain services)
- Booking APIs (single booking entrypoint for all channels)
- Workflow engine (can start synchronous; evolve to queued async where needed)
- PMS adapters + PMS-facing APIs
- AI integrations used by workflows

8) Voice/AI layers (operational intent)

We follow an escalation model:

1. **Lex (primary + cheapest path)** handles:
   - Fixed FAQs and known intents
   - Booking/rescheduling/cancel flows (via Booking APIs)
2. **Low-confidence / complex cases** escalate:
   - **SLM** (e.g., "Nova Micro" class) when Lex confidence is low, but a fast/cheap model may resolve
   - **Premium LLM** only for complex cases that require deeper reasoning
3. Optional human handoff remains available as a safety mechanism.

9) External systems & deployments (high level)

- Amazon Connect + Lex for inbound calls
- FastAPI service for booking + workflows
- Postgres as primary datastore
- Object storage for documents/images (later phases)
- Monitoring, security, and HIPAA-aligned controls end-to-end (auditing, encryption, least privilege)

10) Near-term implementation approach (migration vs rewrite)

Recommended approach is **incremental extraction and reuse**, not a full rewrite:

- Keep the existing Oryx integration logic conceptually as the first adapter (`OryxAdapter`)
- Move booking business logic behind the Booking Service interface (in FastAPI)
- Keep Next.js UI as-is where possible; adjust to call FastAPI via BFF endpoints
- Keep voice Lambda/contact flows and update endpoints/secrets as needed to target the new booking API surface